Lab 3:

Scenario: The Hospital Supply Courier

A hospital campus has several departments connected by internal corridors, represented as a graph. An autonomous courier robot moves medical supplies between departments. You are asked to build and evaluate a small navigation module for it.

Specification:

1. Represent the hospital's departments and corridors as a graph (a dictionary), with at least 8 departments and enough connections that at least two different routes exist between at least one pair of departments. Design this graph yourself.

2. Formally state the search problem formulation (State, Initial State, Actions, Goal Test) for navigating this graph, in a markdown cell.

3. Implement a `bfs_route(graph, start, goal)` function, using `collections.deque`, that returns the shortest path (in terms of number of corridors traveled) between two departments, along with the number of nodes explored.

4. Implement a `dfs_route(graph, start, goal)` function, using a plain `list` as a stack, that returns a valid (not necessarily shortest) path between the same two departments, along with the number of nodes explored.

5. Run both functions on two different start/goal pairs of your choice, printing the path, path length, and nodes explored for each.

6. In a markdown cell, answer the following in 4–6 sentences total:

   For at least one of your two test pairs, did BFS and DFS return routes of different lengths? Explain why or why not, referencing the specific structure of your graph.

   The hospital wants to add a rule: "the courier must avoid Department X between 12pm–1pm due to cleaning." Without writing code, explain in 2–3 sentences how you would need to modify the search problem's Actions or successor function to support this — would BFS/DFS themselves need to change, or just the problem formulation?